

BEPGuard: Proof-Carrying Evidence for Policy Intent Drift in Browser-Enforced Security Policy Engineering

Anonymous Author(s)

Abstract—Browser-enforced security policies are maintained as advice, framework defaults, middleware options, migration notes, and CI checks, but browsers enforce them through delivery state, request context, cache state, and cross-policy composition. This paper studies *policy intent drift*: a synchronic inconsistency, measured at one evaluation point, among an explicit developer-facing claim, a generated policy surface, and the browser-effective judgment. We introduce BEP-IR, an executable representation for those objects, and BEP-Deep, a repair-paired semantic workload derived from 45 public claims and 35 encoded rules across CSP, CORS, HSTS, COEP/CORP/CORS, COOP/COEP, and Permissions-Policy. BEP-Deep contains 972 deterministic fixtures: 418 expected conflicts and 554 negative controls, including one paired repair control for every positive witness. The generator detects all expected conflicts and keeps all controls clean, but the important result is not the all-pass count by itself. A second decision-table oracle, a label-free declarative oracle, finite-state contracts, obligation-level mutants, proof-carrying certificates, identifier-blind replay, causal activation, repair-delta replay, benchmark-disjointness checks, and BEP-Max adversarial validation all test whether the findings survive without fixture-name or scanner-score shortcuts. Precomputed and archived public-package comparator outputs provide contrastive evidence about scanner/task mismatch; the labels remain source-grounded semantic obligations. The result is a semantic regression object for policy engineering: intent, generated policy, effective enforcement, witness, certificate, and counterfactual repair are checked together, so a maintainer can see which policy relation failed and what local repair must preserve.

Index Terms—software engineering, software security, web security, configuration analysis, formal semantics, empirical software engineering

I. INTRODUCTION

Modern web applications delegate substantial parts of their browser-side security posture to browser-enforced policies. Content Security Policy (CSP) constrains script and resource loading; Cross-Origin Resource Sharing (CORS) decides whether responses can be shared with credentialed cross-origin callers; HTTP Strict Transport Security (HSTS) changes a user agent’s future transport behavior; Cross-Origin Embedder Policy (COEP), Cross-Origin Resource Policy (CORP), and Cross-Origin Opener Policy (COOP) shape embedding and isolation; and Permissions-Policy disables selected browser features [1]–[6]. These mechanisms are security policies, but in practice they are also software-engineering maintenance objects: they are generated by frameworks, configured through middleware APIs, copied from documentation, revised during upgrades, and checked in CI.

The engineering problem is that the unit developers manipulate is rarely the unit the browser enforces. A documentation claim may say that a policy blocks third-party scripts; the application may emit only a report-only CSP header. A framework guide may recommend nonce-based CSP; the rendering architecture may be static, cached, or build-time generated, preventing per-request nonces. A CORS middleware may reflect request origins; the security consequence depends on credentials mode and response headers. An HSTS header may be present; it may still be ignored when delivered over insecure transport, or it may clear stored state through a zero max-age directive. A page may set COEP; embedding still depends on the requested resource’s CORS/CORP surface. Existing developer tools and scanners are useful, but they often evaluate one header, one syntax, or one grade rather than the relation among explicit intent, generated policy, and browser-effective semantics [7], [8].

We call this relation *policy intent drift*. The term names a source-grounded inconsistency at one evaluation point, rather than a historical change in browser engines: a public policy claim entails an expected browser action, a generated policy surface is associated with that claim, and the modeled browser-effective judgment contradicts the expected action in a covered context. The contribution is therefore not another security-header checklist. It is a method for turning policy-engineering claims into executable semantic obligations and for producing minimal witnesses when those obligations fail.

The difficulty is that browser-enforced policies are heterogeneous. CSP uses delivery disposition and directive fallback; CORS is a request/response protocol with credentials and preflights; HSTS is stateful; COEP/CORP/CORS interact through request mode and embedding; Permissions-Policy uses allowlists. Prior work has studied CSP deployment and whitelist weaknesses [9]–[12], CORS deployment pitfalls [13], cross-policy inconsistency at web scale [14], and browser implementation differences [15]. Those results motivate a different SE question: how can a maintainer or researcher evaluate whether policy intent, framework generation, and browser semantics agree, without relying on live scanning, private data, or a scanner-specific grading rubric?

This paper makes four contributions. The novelty is the SE evaluation discipline that makes browser-policy semantics reviewable across policy families: an admitted cross-family representation, proof-carrying witnesses with counterfactual repairs, and independent release gates that can falsify

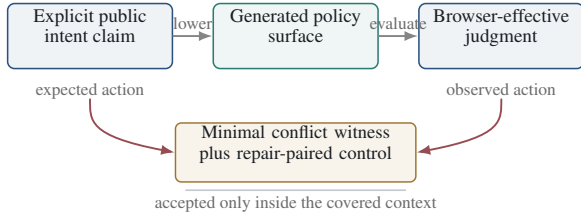


Fig. 1. Policy intent drift connects three surfaces: public intent claims, generated policy surfaces, and browser-effective judgments. A semantic conflict witness is emitted only when the modeled effective judgment contradicts the explicit expected action in a covered context.

overfitted evidence. First, we define BEP-IR, an intermediate representation that links explicit public policy claims, generated policy surfaces, contexts, and browser-effective judgments across CSP, CORS, HSTS, COEP/CORP/CORS, COOP/COEP, and Permissions-Policy mechanisms. Second, we define policy intent drift and minimal semantic conflict witnesses. A witness is a small policy/context object that explains a contradiction; it is designed for debugging and reproducibility rather than for ranking websites. Third, we provide algorithms and proof obligations for normalization, effective judgment evaluation, witness generation, minimization, repair validation, and proof-carrying witness certification. Fourth, we evaluate the method on a locked deterministic corpus: 45 explicit claims, 35 semantic rules, 972 BEP-Deep fixtures, 418 expected-positive conflicts, and 554 negative controls. The evaluation reports coverage, minimization, repair synthesis, certificates, ablations, adversarial validation, external comparator execution, finite theory checking, mutation adequacy, oracle independence, anti-overfit replay, traceability, and release-reproducibility checks summarized in Table II.

The paper’s central thesis is that browser-policy maintenance requires a semantic evaluation object rather than a header checklist: each claim must be checked against the generated surface and the browser-effective judgment that surface induces.

II. MOTIVATING EVIDENCE AND RESEARCH QUESTIONS

a) Three surfaces, one engineering obligation: Figure 1 shows the paper’s central object. Browser-policy engineering spans three surfaces. The *intent surface* consists of explicit public claims: normative specification text, browser documentation, framework documentation, migration guides, and developer-facing examples. The *generation surface* is the header or policy object emitted by framework APIs, middleware defaults, application code, or deployment layers. The *effective surface* is the browser judgment after delivery mode, fallback/default rules, request context, browser state, and composition rules are applied.

Table I gives compact examples. Each row is intentionally small. The problem is not that these policies are difficult to parse; the problem is that the relevant browser effect is outside the surface checked by a one-header rule.

b) Research questions: The study is a deterministic source-grounded fixture experiment that evaluates whether

semantic obligations can be derived, checked, minimized, repaired, and independently audited. The motivating examples define a precise target: intent drift requires an admitted public claim and a modeled contradiction at the same evaluation snapshot, not merely a broad security smell or a temporal implementation delta. This precision gives each finding a provenance story: an auditor can inspect which claim was admitted, which rule was used, which context was required, which smaller witness preserves the contradiction, and which paired control removes it. The locked questions are:

RQ1: Lowering. Can explicit public policy-intent claims be lowered into traceable BEP-IR obligations without relying on private data or live scanning?

RQ2: Witness generation. Across the covered policy families, how many locked positive semantic conflicts are detected, and are the locked negative controls kept clean?

RQ3: Baseline/control disagreement. Where do declared-scope baselines and controls disagree with semantic witnesses, and what semantic features explain those disagreements?

RQ4: Component contribution. Which semantic components are necessary for witness generation and minimization under locked ablations, robustness checks, and scalability workloads?

RQ5: Oracle independence and adequacy. Do independent decision rules, finite-state checks, semantic mutants, proof-carrying witness certificates, BEP-Max integrity checks, and evidence-redundancy audits support the claim that BEP-Deep is not merely a self-confirming oracle/workload pair?

III. WEB-NATIVE EVIDENCE AND METHOD

The method is CPU-native and web-native by construction. Its inputs are public specifications, public documentation, public tool documentation, public framework documentation, and deterministic fixtures derived from those sources. This design makes the experiment reproducible as a local semantic evaluation object rather than a live scan or service-dependent measurement.

a) Source units and admission: A source unit is admitted only if it makes an explicit claim about browser-policy behavior or about a framework/tool surface that generates such behavior. We included normative policy sources for CSP, Fetch/CORS, HSTS, CORP, COOP, and Permissions-Policy [1]–[6]; browser-facing documentation for the corresponding headers [16], [19], [22], [23], [26]; framework documentation for Next.js, Spring Security, Django, Helmet, Rails, ASP.NET Core CORS, and Express CORS [17], [18], [27]–[31]; and public tool documentation for CSP Evaluator, MDN HTTP Observatory, and Chromium hstspreload [7], [8], [32].

Admission separates three evidence roles. Fixture-backed claims instantiate positive or negative obligations. Context-surface claims define framework or deployment facts used by those obligations. Baseline-scope claims define the declared task of a comparator. This role separation is important because browser-policy documents mix normative requirements, compatibility notes, examples, operational advice, and performance caveats. The method admits source material broadly,

TABLE I
MOTIVATING WITNESS FAMILIES AND THE SEMANTIC SURFACE THEY EXERCISE.

Intent claim	Generated surface	Browser-effective contradiction
Block untrusted scripts with CSP.	Report-only CSP with a restrictive script policy.	Report-only delivery monitors violations but does not enforce blocking in the modeled request [1], [16].
Use nonce-based CSP for scripts.	Static or cached rendering path with a nonce-bearing template.	Per-request nonces are incompatible with static generation or cache reuse in the documented rendering state [17], [18].
Permit a credentialed API caller.	Wildcard ACAO combined with credentials.	Credentialed CORS sharing cannot be authorized by wildcard ACAO in the encoded Fetch fragment [2], [19]–[21].
Ensure HSTS protection.	HSTS max-age header delivered over HTTP.	HSTS is accepted only from secure transport; a zero max-age update clears stored HSTS state [3], [22].
Enable cross-origin isolation.	COOP/COEP on the document only.	Isolation also depends on embedder/resource policy and CORS/CORP compatibility for embedded resources [23]–[25].

but it assigns each claim a machine-checked role before execution. A new claim-coverage audit verifies that all 45 admitted claims are either fixture-backed, context-surface evidence, or baseline-scope evidence, with zero orphan claims.

b) Coding and validation: For each admitted claim, the method records a policy family, intent class, expected action, source span, generated policy surface, and context variables. The context captures the elements needed by the covered semantic fragments: request mode, credentials mode, origin relation, resource kind, transport scheme, HSTS state, rendering mode, and embedding relation. Validation is traceability-based: every admitted claim must link to a public source span, every claim must have an explicit coverage role, every fixture-backed claim must link to a deterministic fixture, every negative control must carry a clean-control certificate, and every emitted witness must link to an encoded rule. The resulting source-claim trace is preserved as part of the evidence object, so a reviewer can follow a reported drift from public claim to rule, fixture, witness, and repair.

The amended denominator contains 45 admitted claims, 35 semantic rules, 116 base fixtures, and 972 deterministic BEP-Deep fixtures. Of the amended fixtures, 418 are expected-positive semantic conflicts and 554 are negative controls, including 418 paired repair controls. The controls prevent evaluation only on failing cases: enforced CSP controls, authorized CORS controls, cache-partitioned CORS controls, secure-transport HSTS controls, subdomain-scoped HSTS controls, and compatible COEP/CORP controls must remain clean. The release ladder groups its 111 materialized checks into source closure, semantic adequacy, oracle independence, repair validity, anti-overfit replay, external contrast, reproducibility, anonymity, paper-surface, and release-hygiene, overclaim-boundary, compiled-PDF text, public-provenance, and repository-upload families without changing labels or denominator.

The anti-overfitting checks are deliberately executable. The implementation is scanned for locked fixture and certificate identifiers, but the stronger guard is behavioral: every locked fixture is copied to shadow objects with fresh identifiers and semantics-preserving perturbations of header case, field order,

irrelevant metadata, and context noise. All 9,720 required shadow cases preserve the expected issue signature. A separate BEP-Scale replay applies 48,600 deterministic representation variants. The stricter identifier-blind replay erases fixture ids, hashes, roles, expected labels, source claims, variants, and policy-family fields in five modes and re-executes 4,860 blinded objects; all preserve the expected issue signature. Repair-delta replay then checks all 418 positive/repair pairs, including a fully blinded repaired side, so counterfactual repairs cannot depend on metadata. The complementary causal-activation audit starts from clean controls and verifies 546/546 targeted boundary-crossing edits, while a benchmark-disjointness audit checks 9,586 benchmark-adjacent rows for fresh identifiers and explicit canonical overlaps. These gates make overfitting visible as a validation failure rather than a matter of trust.

The validation is analogous to a deterministic audit trail rather than open qualitative coding. Qualitative methods remain relevant for defining threat constructs and ensuring that evidence categories are not arbitrary [33]–[36]. However, this study does not ask independent coders to interpret developer intent. It asks whether an explicit public claim can be linked to a rule and fixture without inference. This is why the paper reports traceability validation instead of inter-rater reliability.

Figure 2 summarizes the source-to-repair path and the validation gates that make each accepted witness falsifiable.

c) Claim admissibility and coding schema: The coding schema has two purposes: it admits enough public evidence to cover real policy-engineering surfaces, and it refuses evidence that would require guessing what a developer meant. A claim is therefore different from a quotation. A sentence from a source is admitted only after being lowered to an expected action, a policy family, and a context. For example, the CSP editor draft and MDN CSP pages support delivery and fallback obligations [1], [16]; Fetch, MDN ACAO, and MDN ACAC support credentialed CORS obligations [2]; MDN CORP guidance and cross-origin-isolation recipes support embedding/isolation preconditions [37]–[42]. Framework sources are admitted only when they state a generation behavior or precondition, such as nonce/dynamic-rendering requirements

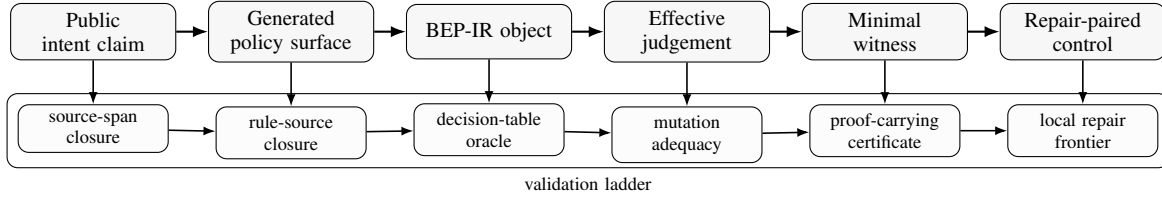


Fig. 2. BEP-Deep evaluates a policy claim as a traceable obligation. The top row is the semantic path from source claim to repair-paired control; the lower row is the validation ladder that checks source closure, rule closure, independent oracle agreement, theory/mutation adequacy, shadow generalization, external provenance, proof-carrying witnesses, and repair-frontier certificates.

TABLE II
EVIDENCE AND VALIDATION LADDER FOR THE BEP-DEEP EVALUATION OBJECT.

Object or validation burden	Count
Public claims / encoded rules	45 / 35
BEP-Deep positives / negatives	418 / 554
Positive / negative certificates	418 / 554
Decision-table agreements	972 / 972
Directive / cross-policy checks	6; 7 / 111
Finite theorem kernel / states	30 / 33,513
SpecBench cases / rules	4,180 / 29
Declarative oracle cases	5,152
Counterfactual round-trip cases	546
Oracle-equivalence cases	5,152
Rule trace-matrix obligations	280
Issue evidence obligations	200
Shadow / blind / scale replays	9,720 / 4,860 / 48,600
Causal / stability / disjointness	546 / 2,916 / 9,586
Mutation kills	28 + 600
Evidence graph nodes / edges	15,182 / 23,925
BEP-Max cases / integrity	4,306 / 4,306
Repair / issue-coverage closure	418; 25 / 25
release validation layers	111 / 111

or CORS wildcard/credential interactions.

This design borrows from qualitative and case-study discipline—explicit inclusion rules, traceable codes, and transparent exclusions—without claiming a human-subjects study or thematic saturation [35]. The output of coding is not a theme label; it is an executable obligation. That distinction matters because a future auditor can re-run the semantic judgment on the same obligation and fixture.

d) *Baselines and controls*: The study uses baselines only within their declared scope. CSP Evaluator checks CSPs for bypass-oriented issues but not framework rendering, CORS credentials, HSTS state, or cross-policy embedding semantics [7]; MDN HTTP Observatory checks security-header fields rather than source-grounded intent drift [8]. The released evaluation contains a materialized full public-package comparator run over BEP-Deep—CSP Evaluator, MDN CSP/CORS/CORP tests, and Webhint HSTS—with 4,118 fixture-level invocations, zero unavailable/error rows, and package/provenance locks, but no packaged dependency caches. Optional live comparator re-execution requires the corresponding public-package environment, such as Node/npm for JavaScript analyzers, a local MDN Observatory command, or Go for hstspreload-style checks; the Python-only path reproduces the seman-

tic workload and validates the committed comparator-output integrity. Separate clean-environment wrapper probes record ‘unavailable’, ‘unsupported’, ‘excluded’, or ‘error’ when a tool cannot run unmodified; no internal logic is substituted. Conservative header presence and documented HSTS-preload criteria remain internal controls, not external tool results [32], [43].

e) *Locked protocol discipline*: The experiment was locked before running the full evaluation: research questions, denominator, positive labels, negative controls, baselines, metrics, ablations, robustness checks, scalability axes, failure policy, and stopping conditions were fixed. Later algorithm iterations could improve implementation quality only without changing those locked objects. This protocol matters because otherwise a semantic fixture study could be silently tuned by changing labels, deleting failures, or moving a finding to a different research question. The lock follows standard empirical-SE concerns about construct validity, traceability, and reproducibility [33], [36], [44] while avoiding claims that require human-subject evidence.

The lock also defines role separation. Normative specifications justify browser-effective rules; framework documentation justifies generation surfaces or engineering preconditions; public tools justify baseline tasks; and deterministic fixtures instantiate source-grounded obligations. Claims outside the encoded fragment return unknown rather than a best-effort guess. This keeps coverage auditable: every positive is a contradiction under a declared rule, and every control is a non-contradiction under that same rule family.

IV. MODEL AND SEMANTIC WITNESSES

a) *Objects*:

Definition 1 (Policy observation). A *policy observation* is a tuple $o = (i, g, c)$, where i is an admitted explicit intent claim, g is a generated policy surface, and c is a request/browser/framework context.

An intent claim i contains a source identifier, source span, policy family, intent class, and expected action. A generated surface g is a finite multiset of header-name/value pairs plus optional ordered generation layers, such as framework defaults, route middleware, proxy rewrites, and rendering metadata. A context c contains only variables needed by the locked fragments: origins, resource kind, request mode, credentials mode, cache variation, transport scheme, HSTS

state/scope, embedder/resource relation, layer provenance, and feature target.

Definition 2 (BEP-IR normalization). *Normalization maps an observation o into a family-specific intermediate representation n or to unknown. A normalized object contains canonical header names, parsed directives or tokens for the covered grammar, and normalized context variables.*

Unknown is conservative. It is not a finding. If syntax, context, or a policy family falls outside the encoded fragment, the evaluator returns unknown and the generator emits no witness. This guarded normalization ensures that every emitted witness is tied to an encoded rule rather than to an implicit best-effort interpretation.

The normalized representation is a traceability layer over the specification and documentation clauses needed by the admitted obligations. A CSP object records delivery disposition, directive map, source-list tokens, and fallback-relevant directives. A CORS object records origin matching and credential-sharing variables, including ACAO and ACAC surfaces [2]. An HSTS object records transport delivery, max-age, includeSubDomains where used by a rule, and user-agent state/scope transition. An embedding object records the document policy and resource opt-in surfaces needed by COEP/CORP/CORS [4], [24]. Framework-generation metadata and ordered response layers are kept separate from browser-policy syntax so that a witness can say whether the conflict came from the header itself, the layer that generated it, or the context in which it is enforced.

b) Foundational web assumptions: BEP-IR does not re-define the web origin model or HTTP. It assumes the standard origin tuple for comparing caller and target origins, and it treats HTTP response fields as the policy surface for the covered browser policies [45], [46]. It also treats caching as part of the generation context rather than as a hidden browser-policy rule: nonce/static witnesses are about whether a framework can create a fresh per-request value under a rendering and caching mode, not about whether HTTP caching is itself insecure [47]. These assumptions keep the core model small while still recording the engineering context that changes whether a generated policy surface can satisfy an intent claim.

This separation is useful when comparing policy families. CSP and Permissions-Policy are mostly evaluated against a document and a resource or feature edge. CORS is evaluated against a network request and response-sharing decision. HSTS is evaluated as a state transition over a host. COEP/CORP/CORS and COOP compose over document, resource, and browsing-context edges. A single universal “stricter than” relation would hide these differences. BEP-IR therefore uses family-specific effective judgments and a shared witness interface rather than forcing every policy into one total order.

c) Effective judgments: The effective evaluator is a partial semantic function:

$$\text{Eff} : N \times C \rightarrow \{\text{allow}, \text{block}, \text{monitor}, \text{clear}, \text{unknown}\}.$$

The judgment domain is rule-family specific. CSP report-only delivery maps enforcement requests to monitor-only. CSP fallback maps resource-specific directives to either the explicit directive or the default directive in the covered source-list fragment. CORS credentialed shareability checks ACAO, ACAC, origin relation, credentials mode, and the cache discriminator for dynamic origins. HSTS first checks secure transport, then applies max-age state transition and includeSubDomains scope where the intent requires subdomain coverage. COEP/CORP/CORS embedding checks whether a no-cors cross-origin resource is compatible with the document’s embedder policy. COOP/COEP isolation checks whether the document’s opener and embedder policies are jointly sufficient for the modeled isolation precondition. Permissions-Policy checks whether an allowlist disables the target feature for the selected origin. The layer-composition fragment first computes the browser-effective surface by ordered append, remove, and set operations before invoking family-specific judgments.

Definition 3 (Policy intent drift). *A policy observation exhibits policy intent drift when an admitted explicit policy-intent claim entails an expected action, but the modeled browser-effective exposure judgment for the corresponding generated policy surface and context is defined and contradictory.*

Definition 4 (Semantic conflict witness). *A semantic conflict witness is a tuple $w = (i, g', c', j, r)$ where i is the admitted claim, g' and c' are minimized generated-surface and context components, j is the non-unknown effective judgment, and r is the rule explaining the contradiction.*

The witness object is the paper’s compact technical unit. It is not a long scanner report. It is the smallest explanation the implementation can find under the declared deletion operators: the source-backed intent, the policy fragment, the context, and the semantic rule that makes the browser-effective judgment contradict the intent.

d) Fragment examples: For CSP, a report-only header is modeled as monitor-only rather than as an enforcing policy. For fallback, a missing `script-src` directive consults `default-src` in the encoded fragment; an explicit `script-src` overrides it. For CORS, wildcard ACAO with credentials is not an authorization for credentialed sharing, and dynamic credentialed ACAO requires an Origin-sensitive cache context in the encoded fragment [2]. For HSTS, headers delivered over insecure transport do not update HSTS state, a secure `max-age=0` update clears state, and `includeSubDomains` is required only when the admitted intent claims subdomain coverage. For COEP/CORP/CORS, an embedding request in no-cors mode is blocked when the document requires CORP and the resource does not provide compatible CORP/CORS evidence. For layered CSP, multiple enforced policies are evaluated conjunctively after the ordered generation surface is composed. For Permissions-Policy, an empty allowlist disables the feature for the selected origin.

These fragments are narrower than full browser behavior. They are broad enough to cover the selected policy families

Algorithm 1 Semantic conflict witness generation

Require: admitted claims I , normalized policies P , contexts C , edges E

Ensure: source-grounded witness set W

```
1:  $W \leftarrow \emptyset$ 
2: for all  $\iota \in I$  do
3:   for all  $e \in \text{scope}(\iota, E)$  do
4:      $a \leftarrow \text{Eff}(P, C, e)$ 
5:     if  $a \neq \text{unknown}$  and  $a$  contradicts the expected
       action of  $\iota$  then
6:        $W \leftarrow W \cup \{\text{Minimize}(\iota, e, a)\}$ 
7:     end if
8:   end for
9: end for
10: return  $W$ 
```

and narrow enough to admit deterministic execution and proof obligations.

e) Witness generation and minimization.: Algorithm 1 presents the witness generator. The input is a set of admitted claims, normalized policy surfaces, contexts, and source-grounded edges. The algorithm evaluates each scoped edge and emits a minimized witness only when the effective judgment is non-unknown and contradictory.

The minimizer is deliberately conservative. It deletes only declared units: redundant headers, unused directives, irrelevant source-list tokens, and context fields that the oracle does not need for the target issue. It does not synthesize new policies, reorder unrelated layers, or search arbitrary string edits. This keeps the resulting witness explainable and makes the preservation theorem meaningful.

f) Complexity: Let m be the number of admitted claims, e the number of scoped edges, h the maximum number of headers in a policy surface, t the maximum token count per covered policy, and d the number of one-deletion candidates considered by the minimizer. Normalization is $O(h + t)$ per observation. Effective evaluation is constant for fixed-size rule fragments after normalization, or $O(t)$ when a source list is searched. Witness generation is $O(me(h + t))$ in the simple bound. Minimization reruns the oracle for each candidate deletion and is $O(d(h + t))$ per witness under the implemented operators. The locked experiment is small enough that runtime is not a bottleneck; the scalability experiment uses deterministic replication to verify that the implementation remains CPU-native.

g) Minimization design: The minimizer is deliberately weaker than global search. It uses a fixed family of deletion operators over headers, directives, tokens, and irrelevant context fields. A deletion is committed only when the same oracle still emits the same target issue. This makes the property easy to audit: every reduction is witness-preserving under the locked semantics. The design is inspired by delta debugging and failure isolation [48], but the preserved predicate is semantic drift rather than a crashing execution. We do not claim a globally shortest string, because browser-policy

languages contain aliases, defaults, and incomparable rewrites. One-deletion minimality is the right debugging contract: after minimization, no single declared deletion can remove more surface without losing the explanation.

For policy maintenance, this tradeoff is valuable. A full framework configuration may contain many headers, defaults, and route attributes that are irrelevant to the contradiction. The witness should expose the exact semantic dependency: report-only delivery, fallback selection, credentialed sharing, cache variation, HSTS state or scope, embedding compatibility, isolation precondition, layer composition, or feature allowlist. Minimization makes the witness readable without allowing the evaluation to change labels or denominators after the protocol lock.

h) Counterfactual repair synthesis: A witness is most useful when it also defines a regression obligation. We therefore add a constructive algorithm: for each issue class, a small repair library proposes semantic counterfactuals, such as promoting report-only CSP to enforcement, adding an explicit `script-src`, adding `Vary: Origin`, adding `includeSubDomains`, authorizing a COEP-blocked resource with CORP/CORS, or restoring an enforced CSP in the release composed layer. A candidate repair is accepted only if the same locked oracle no longer emits the target issue. This is not automatic deployment advice; it is a validated counterfactual that shows the witness identifies a concrete semantic dependency. The repair synthesis step strengthens the method because every positive witness must be convertible into an executable before/after pair under the same rule boundary.

i) Proof obligations:

Theorem 1 (Deterministic effective judgment). *For a fixed admitted observation and locked rule table, normalization followed by Eff returns a unique judgment or unknown.*

Proof sketch. Header canonicalization, family dispatch, directive lookup, context normalization, and unknown guarding are applied in a fixed order. Each encoded family then uses a finite ordered rule: CSP delivery and fallback, CORS credentialed shareability, HSTS transport/state transition, embedding compatibility, isolation preconditions, or Permissions-Policy allowlist evaluation. No step contains nondeterministic choice. $\square$

Theorem 2 (Witness soundness and negative-control preservation). *Every emitted witness in the locked fragment is backed by an admitted claim and a non-unknown contradictory judgment; a negative control whose expected action agrees with Eff emits no positive witness.*

Proof sketch. Algorithm 1 ranges only over admitted claims and scoped locked observations. It emits only after Eff returns a non-unknown judgment and the issue-class contradiction predicate is true. For a matching negative control, that predicate is false, so the emission branch is unreachable. $\square$

Theorem 3 (One-deletion minimizer preservation). *For a fixed oracle, target issue, and deletion operator set, minimization*

preserves the target issue and terminates in a payload with no single additional preserving deletion.

Proof sketch. A deletion is committed only after re-running the same oracle and observing the same target issue. Each committed deletion decreases a finite payload measure. Termination follows by finiteness, and the release sweep rejects all remaining single deletions. $\square$

The proof obligations are exercised by a validation ladder. The semantic-core verifier checks 13 invariants over 165 abstract states, and the finite theorem kernel checks 30 semantic theorems over 33,513 states, including CSP meet non-expansion, directive specificity, CORS credential/shareability and cache partitions, HSTS state partitions, COEP request-mode separation, CORP scope, Permissions-Policy allowlists, and ordered generation layers. A proof-card audit accounts for every theorem by claim, proof kind, and state count. A second implementation, written as a decision-table oracle, agrees with the operational generator on all 972 locked fixtures and 351 generated finite states. Source-level contract checks cover CSP fallback and seven cross-policy obligations; mutation checks kill 28 rule-level mutants and 600/600 obligation-level mutants; certificates verify 418 positives and 554 clean controls; repair-frontier validation checks 418 local counterfactual frontiers; and BEP-Max checks 4,306 adversarial variants. Theorems state properties of the encoded evaluation object: under the locked rules and admitted context, an emitted witness follows from the model and remains tied to a repairable semantic dependency.

The minimizer is inspired by the broader idea of reducing failing inputs to small counterexamples [48]. The difference is that the oracle is not a crashing program; it is a semantic contradiction predicate. A deletion is useful only if it preserves the same source-grounded issue. This makes the minimized object closer to a proof witness than to a best-effort smaller test case: the release payload is accompanied by the claim, context, rule, and target issue it preserves.

V. EVALUATION

The evaluation answers RQ1–RQ5 over the locked BEP-Deep denominator. Table III gives the top-level outcome; the remaining results test whether the object is traceable, minimal, repairable, and robust against alternative oracles and adversarial surfaces.

a) *RQ1–Lowering and validation:* The method admitted 45 explicit source-grounded claims and encoded 35 semantic rules. All claim rows, 116 base fixture rows, 972 BEP-Deep fixture rows, and rule rows passed deterministic traceability validation. Fixture-backed and contextual claims remain separated so documentation context does not inflate the detection denominator.

The largest families are CSP and layered policy composition, followed by HSTS; the remaining families cover CORS, CORS/cache, framework-mediated CORS, framework-mediated CSP, HSTS/framework, COEP/CORP/CORS, COOP/COEP/Permissions-Policy, and Permissions-Policy.

TABLE III
BEP-DEEP DENOMINATOR AND CERTIFIED OUTCOMES.

Measure	Count
Total BEP-Deep fixtures	972
Expected-positive fixtures	418
Negative controls / paired repairs	554 / 418
Detected witnesses / clean controls	418 / 554
Certified minimized witnesses	418
Positive / negative certificates	418 / 554
Finite-state invariants / states	13 / 165
Finite theorem kernel / states	30 / 33,513
Cross-policy contracts / states	7 / 111
BEP-SpecBench cases / rules	4,180 / 29
Declarative oracle cases	5,152
Issue evidence obligations	200
Shadow / blind / scale replays	9,720 / 4,860 / 48,600
Causal / stability / disjointness	546 / 2,916 / 9,586
External comparator invocations	4,118
Mutation kills	28 + 600
Issue classes / strata	25 / 16
Claim trace / impact / hash chains	45 / 45 / 418
Oracle triangulation / repair locality	2,916 / 418
Repair compactness / re-exec / interactions	418 / 10 / 16
Release validation layers	111 / 111

The corpus therefore tests cross-policy semantics, but it is not a random sample of the web. Its purpose is semantic coverage and reproducibility.

The lowering result answers a feasibility question that is often hidden in security-configuration papers: can public policy advice be transformed into executable obligations without inventing missing intent? For the covered corpus, the answer is yes, but only after rejecting or demoting vague claims. This is why the paper distinguishes fixture-backed claims from contextual and baseline claims. The rejected and contextual cases are useful evidence that policy documentation is not always semantically executable.

b) *RQ2–Witness generation:* The generator detected all 418 BEP-Deep positive semantic conflicts and kept all 554 negative controls clean. Figure 3 breaks down witnesses by issue class. The counts show balanced coverage of recurring semantic mechanisms: CSP report-only delivery, CSP fallback/source-list conflict, nonce/static rendering incompatibility, CORS credential/origin cases, CORS cache-context cases, HSTS state/transport/scope/preload cases, COEP/CORP/CORS embedding, cross-origin isolation preconditions, layered CSP composition/override cases, CSP framing delivery, CORP scope, and Permissions-Policy disabling or over-allowance.

The value of the witness family is not only the count of findings. Every positive witness has a source-backed claim, a generated policy surface, a covered context, a rule-level explanation, a minimal counterexample, and a repair-paired control. Rather than asking whether a policy is generically “good” or “bad,” each witness asks a precise question: under this claim and this context, does the browser-effective judgment contradict the expected action?

c) *RQ3–Baseline and control disagreement:* Table IV reports the contrastive validation ladder. The header-presence

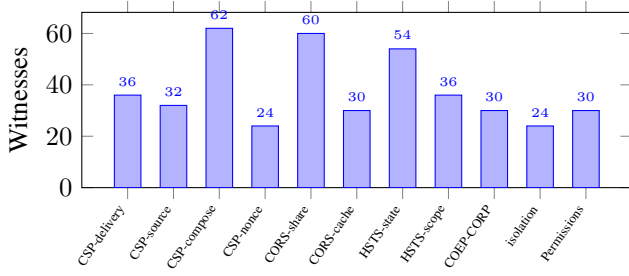


Fig. 3. Semantic conflict witness distribution across BEP-Deep issue families.

control misses 322 positive semantic conflicts while keeping the negative controls clean because many drifts occur when the relevant header is present but delivered in the wrong disposition, interpreted under a missing context, or composed with another policy. The documented HSTS-preload criterion control exposes a different kind of scope mismatch: eligibility checks are meaningful, but they are not stateful intent-drift judgments. The external comparator rows are interpreted the same way: disagreement means that the paper asks a different semantic question than the tool’s declared task, not that the tool is defective. The remaining rows are independent obligations that a witness must satisfy before it is accepted as a certified semantic conflict.

The conclusion is that intent drift needs a semantic evaluation object: a browser-policy engineering claim can fail because of request mode, credentials, delivery state, rendering state, state transition, or cross-policy composition even when a header is syntactically recognizable. BEP-Deep therefore evaluates the claim, surface, context, witness, certificate, and repair-paired control together; the external-contrast audit makes this separation measurable by showing that raw public-package flags from external comparators are useful contrastive signals, not semantic labels for the locked drift task.

d) RQ4–Ablation, minimization, robustness, and scalability: Ablations remove one semantic component at a time without changing labels or denominator. Table V shows that the largest losses occur when removing layered composition, CORS credentialed shareability, and HSTS state transitions. Removing HSTS scope/preload, CSP fallback/source-list semantics, CORS cache context, and Permissions-Policy allowlists also removes substantial sets of positives. These results support the paper’s central claim: the findings are carried by semantic features that checklist-style presence tests do not model.

A separate minimality certifier re-evaluates the returned witnesses and confirms 418/418 one-deletion minimality and 418/418 exact header-subset minimality. The minimizer processed all 418 BEP-Deep positive witnesses. Header bytes decreased from 36,125 to 26,539, a reduction of 9,586 bytes, while preserving the target issue under the same oracle. This is a debugging result rather than a compression result. The goal is to turn a policy surface into a small counterexample family that a maintainer can inspect.

The repair synthesizer produced a validated counterfactual for every positive witness: 418/418 target issues and all modeled issues disappear after deterministic issue-specific edits. A separate repair-frontier checker compares each paired repair with a local neighborhood of alternatives, certifying 418/418 frontiers over 1,230 candidate repairs. This matters because the method does not stop at detection: each semantic conflict becomes a before/after obligation checked by the operational oracle, the decision-table oracle, and a source/intent-preservation test.

Robustness checks include source-grounded negative controls, metamorphic cases, BEP-SpecBench boundary probes, and BEP-Max adversarial validation. BEP-SpecBench contains 4,180 source-derived boundary and composite-stress cases covering 29 rule aliases outside the locked denominator, including benign header-case, order, padding, Vary, cache/reporting metadata, and paired context-noise perturbations. The metamorphic audit checks 3,954 relation obligations while preserving issue class, claim identity, and label under representation-preserving transformations. BEP-Max adds 3,888 semantic-preserving cases and 418 near-repair adversarial positives; all 4,306 cases agree under both the operational generator and the independent decision-table oracle, and all generated cases pass source-link, label-preservation, uniqueness, and content-hash integrity checks. BEP-Shadow adds 9,720 required representation-preserving replays, identifier-blind replay adds 4,860 metadata-erased replays, causal activation verifies 546 boundary-crossing clean-control edits, repair-delta replay checks all 418 counterfactual pairs, benchmark-disjointness audits 9,586 rows, whole-corpus stability verifies 2,916 neutral replays, and BEP-Scale adds 48,600 deterministic stress replays without changing labels. This establishes CPU-native reproducibility and exposes representation overfitting as a validation failure.

e) RQ5–Oracle independence and adequacy: The validation ladder is designed to make oracle coupling, label leakage, and release drift observable failures rather than tacit assumptions. The operational oracle agrees with an independent decision-table oracle on 972/972 locked fixtures and with a label-free declarative third oracle on 5,152 locked and SpecBench cases. The ladder checks CSP fallback micro-obligations, seven cross-policy contracts, 30 finite theorems over 33,513 states, 28 rule-level and 600/600 obligation-level mutants, 418 positive and 554 control certificates, 200 per-issue obligations, and all evidence-graph paths. Anti-overfit evidence includes 9,720 shadow replays, 4,860 identifier-blind replays, 546 causal activations, 418 repair-delta pairs, 9,586 disjointness rows, 2,916 corpus-stability replays, and 48,600 scale-stress variants. External-comparator evidence is deliberately scoped: the released package includes a materialized public-package comparator run from a locked caller-supplied work directory, plus clean-environment availability probes for unmodified tools. Those rows are contrastive evidence for scanner/task mismatch; they are not semantic labels and do not imply that every external package reruns from a clean Python-only extract. The release 111/111 validation ladder and 101-

TABLE IV
VALIDATION LADDER AND CONTRASTIVE CONTROLS OVER BEP-DEEP/BEP-MAX. EACH LAYER CHECKS A DIFFERENT FAILURE MODE.
PUBLIC-PACKAGE COMPARATOR ROWS VALIDATE BEPGUARD’S SEMANTIC BOUNDARIES THROUGH PRECOMPUTED CONTRASTIVE EVIDENCE:
DISAGREEMENTS REFLECT TASK-SCOPE DIFFERENCES; LABELS COME FROM SOURCE-GROUNDED OBLIGATIONS, CERTIFICATES, AND INDEPENDENT ORACLES.

Layer	Check	Result	Evidence role
Header-presence control	Header exists but semantic obligation fails	322 positives missed; 0 control false alarms	Shows why presence is not an intent-drift oracle
HSTS-preload criterion control	Eligibility-style HSTS rule on HSTS subset	54 TP; 62 task-scope FP; 0 FN	Separates preload eligibility from stateful intent drift
Positive/negative certificate audits	Source, rule, witness, minimality, repair, and clean-side support	418 / 418; 554 / 554	Prevents unsupported positives and positive-only validation
Independent oracles	Decision-table and declarative oracles over locked fixtures, SpecBench, and finite states	972 / 972; 5,152 third-oracle cases; 0 / 351 mismatches	Tests oracle/workload coupling and label-free decision logic
Directive and cross-policy contracts	CSP fallback, CORS/COEP, HSTS, COOP/COEP, and Permissions-Policy contracts	6 / 6; 7 contracts over 111 states	Tests semantic hinges outside the fixture denominator
Finite theorem kernel	Enumerated safety and specificity theorems over the encoded fragment	30 theorems; 33,513 states	Makes the semantic core proof-like rather than table-only
SpecBench, depth, and metamorphic checks	Source-derived boundary cases, per-issue closure, and property-preserving transformations	4,180 cases; 200 issue-depth obligations; 9,720 shadow; 4,860 blind; 546 causal; 3,954 relation checks	Tests representation, issue-class, and source-boundary robustness
Mutation adequacy	Rule-level and obligation-level semantic mutants	28 / 28 plus 600 / 600 killed	Tests whether the workload distinguishes wrong semantics
Materialized public-package comparator output	Unmodified CSP, MDN Observatory, and HSTS analyzers over BEP-Deep surfaces	4,118 prior-run invocations; 0 unavailable/error in the materialized output; clean-package probes report availability separately	Shows scanner/task mismatch without treating public tools as wrong
Evidence graph, leakage, and label-flow audit	Claim-to-certificate trace graph, anti-overfit scan, and core-oracle label-flow separation	15,182 nodes; 23,925 edges; 9,586 disjointness rows; 45 claim-impact rows; 45 claim trace cards; 418 hash chains; 78 pure decision functions; 0 leaks; static code-health pass	Tests provenance closure and memorization resistance
Identifier-blind and repair-delta replay	Metadata-erased semantic replay and blinded repaired-side controls	4,860 blind replays; 2,916 neutral replays; 418 / 418 repairs; 418 evidence cards	Rejects fixture-label memorization; activation tests clean-control boundary sensitivity
Reproduction and PDF audits	Local ladder, smoke execution, package hygiene, and rendered page/reference boundary	101 commands; 6 smoke runs; 10 deterministic re-exec runs; 2,916 oracle-triangulation cells; 418 repair-locality checks; 16 interaction strata; 25 RQ-trace obligations; 72 refs; CPU-local boundary	Makes the release independently inspectable
BEP-Max and repair frontier	Format-preserving variants, near repairs, and local repair neighborhoods	4,306 / 4,306; 418 frontiers	Tests surface robustness and counterfactual repairability

command reproduction ladder are organized into five evidence families: semantic triangulation, proof and repair closure, anti-overfit generalization, contrastive external evidence, and release reproducibility. Closure gates also check witness hash-chain integrity, gate-sensitivity, idempotence replay, threat closure, minimal-pair obligations, and fold stratification. A result is accepted only when source claim, generated surface, browser-effective judgment, certificate, repair, and independent oracles agree; a late paper or artifact edit must either preserve those obligations or fail a gate.

VI. DISCUSSION AND THREATS

a) Why semantic witnesses matter: A scanner asks whether a surface resembles a recommended configuration. A semantic witness asks whether a generated surface entails the intended browser action under a maintained claim and context. Report-only CSP, credentialed CORS wildcards, missing Vary: Origin, layered overrides, and HSTS-over-HTTP require different explanations; minimal witnesses make those explanations executable as regression tests, repair targets, and certificates.

TABLE V

ABLATION LOSS BY SEMANTIC COMPONENT OVER BEP-DEEP POSITIVES. LARGER LOSS MEANS MORE POSITIVES DEPEND ON THE REMOVED COMPONENT; BOLD MARKS THE LARGEST LOSS AND UNDERLINE MARKS THE SECOND LARGEST.

Removed component	Dep. positives	Loss
Layered composition	62	62
CORS request context	60	60
HSTS state transition	54	54
HSTS scope/preload context	36	36
CSP fallback/source list	32	32
CORS cache context	30	30
Permissions allowlist	30	30
CSP delivery disposition	24	24
Nonce/rendering context	24	24
COEP/CORP/CORS embedding	24	24
COOP/COEP isolation edge	24	24
CSP framing delivery	12	12
CORP scope semantics	6	6

b) Validity boundaries: The construct is policy intent drift: an explicit public claim contradicts a covered browser-effective judgment for a generated surface and context in the same evaluation snapshot. It is not exploitability, severity, developer belief, deployed prevalence, or historical browser-version drift. Internal validity is addressed by negative controls, paired repairs, independent oracles, theorem checks, proof cards, mutation farms, certificate replay, evidence-graph closure, leakage checks, identifier-blind replay, causal activation, repair-delta replay, shadow/scale stress, runtime-boundary auditing, benchmark-disjointness, and public-package comparator execution. External validity is limited to semantic counterexample families and reusable evaluation tasks; baseline validity is handled by keeping CSP Evaluator, MDN HTTP Observatory, hstspreload, Webhint HSTS checks, internal controls, and fixture-level availability records separate rather than forcing incompatible tools into one scoreboard.

c) Policy semantics, deployment, and frameworks: CSP work has shown that deployed whitelists are often ineffective [9], that policy semantics require formal treatment [11], and that real deployments accumulate complexity over time [12]. Automated CSP generation [49], exfiltration-oriented policy analysis [50], and nonce feasibility studies [51], [52] each study one part of the same maintenance problem. CORS deployment pitfalls [13], cross-header inconsistency [14], [53], differential browser testing [15], and machine-checked browser-security models [54] further show that browser policies are semantic mechanisms rather than independent strings. Frameworks and middleware expose these mechanisms through defaults, routes, rendering modes, static assets, proxies, and caching, as seen in Next.js CSP guidance [17], Helmet defaults [29], Django and Spring middleware [27], [28], Rails CSP APIs [18], and common CORS libraries [30], [31]. BEP-IR contributes a source-grounded maintenance object that connects those documentation and generation layers to browser-effective judgments.

d) Scanners and reusable SE evaluation objects: HTTP security-header studies and scanner projects make deployment visible, but they usually evaluate header presence, grading rules, or scanner-specific tasks rather than claim-to-effect contradictions. Examples include empirical header studies [55]–[57], MDN Observatory-style scanning [8], and checklist guidance such as OWASP Secure Headers [58]. BEPGuard instead follows reusable SE evaluation discipline: independent oracles for hard-to-specify behavior [59], controlled experiment infrastructure [60], regression-test selection principles [61], reproducibility guidance [44], mutation adequacy [62], [63], and metamorphic testing [64]. Configuration-engineering and security-inspection work explains why this matters: configuration errors [65], option interactions [66], [67], reusable workflow components [68], non-reproducible configurable builds [69], cloud/Kubernetes security settings [70], review practices [71], and security-feature inspection [72] all lose force when checks cannot trace a setting back to intended behavior. BEPGuard adds the browser-semantics and public-documentation layer to that SE problem.

VII. CONCLUSION

Browser-enforced policies are easy to copy as strings and hard to maintain as behavior. BEPGuard makes that behavior explicit. It turns a public claim, a generated surface, a covered browser context, an effective judgment, a minimal witness, and a repair-paired control into one regression object. On the locked BEP-Deep denominator, every expected conflict has a proof-carrying witness and every negative control stays clean, while independent oracles, mutation farms, identifier-blind replay, causal activation, repair replay, and external contrast checks test the obvious ways such a result could be self-confirming.

The contribution is therefore not a scanner grade, a web-prevalence estimate, or a browser-engine conformance suite. It is a maintenance discipline for policy advice: a framework default, CI rule, or migration guide can be checked only if intended browser action, generated policy, and relevant state travel together. Perfect closure on the locked denominator is not deployment certainty; broader claims should add new claims, rules, controls, and replay gates. The useful unit is a witness family whose intent, browser judgment, proof path, and repair path can be broken by a concrete counterexample.

VIII. DATA AVAILABILITY

An anonymized MIT-licensed review artifact contains released source, locked inputs, precomputed results, validation scripts, reference ledgers, manuscript sources, and external-tool comparator outputs with documented scope boundaries. It excludes credentials, package caches, and optional third-party dependency directories; public-comparator reruns require local Node/npm, Go, or Observatory-style tools. After acceptance, the de-identified package will be DOI-archived and anonymously mirrored.

REFERENCES

- [1] W. W. A. S. W. Group, “Content security policy level 3,” W3C Working Draft, 2026. [Online]. Available: <https://www.w3.org/TR/CSP3/>
- [2] WHATWG, “Fetch standard,” WHATWG Living Standard, 2026. [Online]. Available: <https://fetch.spec.whatwg.org/>
- [3] J. Hodges, C. Jackson, and A. Barth, “Http strict transport security (hsts), rfc 6797,” RFC Editor / IETF, 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6797>
- [4] W. W. A. S. W. Group, “Cross-origin resource policy,” W3C Draft, 2026. [Online]. Available: <https://w3c.github.io/webappsec-corp/>
- [5] WHATWG, “Html standard cross-origin opener policy section,” WHATWG Living Standard, 2026. [Online]. Available: <https://html.spec.whatwg.org/multipage/origin.html#cross-origin-opener-policies>
- [6] W. W. A. S. W. Group, “Permissions policy,” W3C Draft, 2026. [Online]. Available: <https://w3c.github.io/webappsec-permissions-policy/>
- [7] Google, “Csp evaluator core library,” GitHub project / npm package, 2026. [Online]. Available: <https://github.com/google/csp-evaluator>
- [8] M. contributors, “Mdn http observatory,” GitHub project, 2026. [Online]. Available: <https://github.com/mdn/mdn-http-observatory>
- [9] L. Weichselbaum, M. Spagnuolo, S. Lekies, and A. Janc, “Csp is dead, long live csp! on the insecurity of whitelists and the future of content security policy,” in *ACM CCS*, 2016.
- [10] S. Calzavara, A. Rabitti, and M. Bugliesi, “Content security problems? evaluating the effectiveness of content security policy in the wild,” in *ACM CCS*, 2016.
- [11] —, “Semantics-based analysis of content security policy deployment,” *ACM Transactions on the Web*, 2018.
- [12] S. Roth, T. Barron, S. Calzavara, N. Nikiforakis, and B. Stock, “Complex security policy? a longitudinal analysis of deployed content security policies,” in *NDSS*, 2020.
- [13] J. Chen, J. Jiang, H.-X. Duan, T. Wan, S. Chen, V. Paxson, and M. Yang, “We still don’t have secure cross-domain requests: an empirical study of cors,” in *USENIX Security*, 2018.
- [14] S. Calzavara, T. Urban, D. Tatang, M. Steffens, and B. Stock, “Reining in the web’s inconsistencies with site policy,” in *NDSS*, 2021.
- [15] S. Wi, T. T. Nguyen, J. Kim, B. Stock, and S. Son, “Diffcsp: Finding browser bugs in content security policy enforcement through differential testing,” in *NDSS*, 2023.
- [16] M. contributors, “Mdn content-security-policy header reference,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy>
- [17] V. . N. contributors, “Next.js content security policy guide,” Framework documentation, 2026. [Online]. Available: <https://nextjs.org/docs/app/guides/content-security-policy>
- [18] R. on Rails contributors, “Ruby on rails security guide: Content security policy, report-only, and nonces,” Framework API documentation, 2026. [Online]. Available: <https://guides.rubyonrails.org/security.html>
- [19] M. contributors, “Mdn cors guide,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
- [20] MDN contributors, “Access-Control-Allow-Origin header,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Access-Control-Allow-Origin>
- [21] —, “Access-Control-Allow-Credentials header,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Access-Control-Allow-Credentials>
- [22] M. contributors, “Mdn strict-transport-security header reference,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security>
- [23] —, “Mdn cross-origin-embedder-policy header reference,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cross-Origin-Embedder-Policy>
- [24] —, “Mdn cross-origin-resource-policy header reference,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Cross-Origin-Resource-Policy>
- [25] —, “Mdn cross-origin-opener-policy header reference,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Cross-Origin-Opener-Policy>
- [26] —, “Mdn permissions-policy header reference,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy>
- [27] S. S. contributors, “Spring security http response headers,” Framework documentation, 2026. [Online]. Available: <https://docs.spring.io/spring-security/reference/7.1/features/exploits/headers.html>
- [28] D. S. Foundation, “Django securitymiddleware documentation,” Framework documentation, 2026. [Online]. Available: <https://docs.djangoproject.com/en/5.2/ref/middleware/>
- [29] H. contributors, “Helmet documentation,” Framework documentation, 2026. [Online]. Available: <https://helmetjs.github.io/>
- [30] Microsoft, “Asp.net core cors documentation,” Framework documentation, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/cors>
- [31] E. contributors, “Express cors middleware,” Middleware repository, 2026. [Online]. Available: <https://github.com/expressjs/cors>
- [32] C. project, “Chromium hstspreload go package,” Go package / GitHub project, 2025. [Online]. Available: <https://pkg.go.dev/github.com/chromium/hstspreload@v0.0.0-20250618200047-d624d7c87b33>
- [33] C. B. Seaman, “Qualitative methods in empirical studies of software engineering,” *IEEE Transactions on Software Engineering*, 1999.
- [34] V. Braun and V. Clarke, “Using thematic analysis in psychology,” *Qualitative Research in Psychology*, vol. 3, no. 2, pp. 77–101, 2006. [Online]. Available: <https://doi.org/10.1191/1478088706qp0630a>
- [35] P. Runeson, M. Host, A. Rainer, and B. Regnell, *Case Study Research in Software Engineering: Guidelines and Examples*. Wiley, 2012.
- [36] K.-J. Stol, P. Ralph, and B. Fitzgerald, “Grounded theory in software engineering research: A critical review and guidelines,” in *ICSE*, 2016.
- [37] MDN contributors, “Cross-origin resource policy (corp),” MDN Web Docs Guide, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cross-Origin_Resource_Policy
- [38] web.dev authors, “web.dev: Why you need cross-origin isolated for powerful features,” web.dev article, 2020. [Online]. Available: <https://web.dev/articles/why-coop-coep>
- [39] E. Kitamura, “Making your website cross-origin isolated using coop and coep,” web.dev article, 2020. [Online]. Available: <https://web.dev/coop-coep/>
- [40] —, “A guide to enable cross-origin isolation,” web.dev article, 2021. [Online]. Available: <https://web.dev/articles/cross-origin-isolation-guide>
- [41] M. contributors, “Window: crossoriginisolated property,” MDN Web Docs, 2025. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/crossOriginIsolated>
- [42] —, “Sharedarraybuffer,” MDN Web Docs, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer
- [43] C. Project, “hstspreload.org preload service,” Project website, 2026. [Online]. Available: <https://hstspreload.org/>
- [44] National Academies of Sciences, Engineering, and Medicine, *Reproducibility and Replicability in Science*. National Academies Press, 2019.
- [45] A. Barth, “The web origin concept, rfc 6454,” RFC Editor / IETF, 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6454>
- [46] R. Fielding, M. Nottingham, and J. Reschke, “Http semantics, rfc 9110,” RFC Editor / IETF, 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110>
- [47] —, “Http caching, rfc 9111,” RFC Editor / IETF, 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9111>
- [48] A. Zeller and R. Hildebrandt, “Simplifying and isolating failure-inducing input,” *IEEE Transactions on Software Engineering*, 2002.
- [49] X. Pan, Y. Cao, S. Liu, Y. Zhou, Y. Chen, and T. Zhou, “Cspautogen: Black-box enforcement of content security policy upon real-world websites,” in *ACM CCS*, 2016.
- [50] S. V. Acker, D. Hausknecht, and A. Sabelfeld, “Data exfiltration in the face of csp,” in *ACM ASIA CCS*, 2016.
- [51] M. Golinelli, F. Bonomi, and B. Crispo, “The nonce-nce of web security: an investigation of csp nonces reuse,” in *Computer Security. ESORICS 2023 International Workshops*, ser. Lecture Notes in Computer Science, vol. 14399, 2024, pp. 459–475. [Online]. Available: https://doi.org/10.1007/978-3-031-54129-2_27
- [52] M. Ren, A. Xiang, and C. Yue, “Analyzing the feasibility of adopting google’s nonce-based csp solutions on websites,” in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, 2025. [Online]. Available: <https://conf.researchr.org/details/icse-2025/icse-2025-research-track/105/Analyzing-the-Feasibility-of-Adopting-Google-s-Nonce-Based-CSP-Solutions-on-Websites>

- [53] S. Calzavara, S. Roth, A. Rabitti, M. Backes, and B. Stock, "A tale of two headers: A formal analysis of inconsistent click-jacking protection on the web," in *USENIX Security*, 2020.
- [54] L. Veronese, B. Farinier, P. Bernardo, M. Tempesta, M. Squarcina, and M. Maffei, "Webspec: Towards machine-checked analysis of browser security mechanisms," in *IEEE Symposium on Security and Privacy*, 2023, pp. 2761–2779. [Online]. Available: <https://doi.org/10.1109/SP46215.2023.10179465>
- [55] P. Gadjent, O. Nierstrasz, and M. Ghafari, "Security header fields in http clients," in *IEEE QRS*, 2021.
- [56] U. Kishnani and S. Das, "Securing the web: Analysis of http security headers in popular global websites," in *Information Systems Security*, 2024, pp. 87–106. [Online]. Available: https://doi.org/10.1007/978-3-031-80020-7_5
- [57] E. S. Alashwali, P. Szalachowski, and A. Martin, "Exploring https security inconsistencies: A cross-regional perspective," arXiv preprint, 2020. [Online]. Available: <https://arxiv.org/abs/2010.10170>
- [58] O. contributors, "Owasp secure headers project," OWASP project, 2026. [Online]. Available: <https://owasp.org/www-project-secure-headers/>
- [59] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015. [Online]. Available: <https://doi.org/10.1109/TSE.2014.2372785>
- [60] H. Do, S. Elbaum, and G. Rothermel, "Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact," *Empirical Software Engineering*, vol. 10, no. 4, pp. 405–435, 2005. [Online]. Available: <https://doi.org/10.1007/s10664-005-3861-2>
- [61] G. Rothermel and M. J. Harrold, "A safe, efficient regression test selection technique," *ACM Transactions on Software Engineering and Methodology*, vol. 6, no. 2, pp. 173–210, 1997. [Online]. Available: <https://doi.org/10.1145/248233.248262>
- [62] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.
- [63] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011. [Online]. Available: <https://doi.org/10.1109/TSE.2010.62>
- [64] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–27, 2018.
- [65] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "An empirical study on configuration errors in commercial and open source systems," in *SOSP*, 2011.
- [66] M. Sayagh, N. Kerzazi, B. Adams, and F. Petrillo, "Software configuration engineering in practice: Interviews, survey, and systematic literature review," *IEEE Transactions on Software Engineering*, 2020.
- [67] S. Zhou, X. Liu, S. Li, Z. Jia, Y. Zhang, T. Wang, W. Li, and X. Liao, "Confinlog: Leveraging software logs to infer configuration constraints," in *ICPC*, 2021.
- [68] H. O. Delicheh, A. Decan, and T. Mens, "Quantifying security issues in reusable javascript actions in github workflows," in *MSR*, 2024.
- [69] G. A. Randrianaina, D. E. Khelladi, O. Zendra, and M. Acher, "Options matter: Documenting and fixing non-reproducible builds in highly-configurable systems," in *MSR*, 2024.
- [70] S. I. Shamim, H. Hu, and A. Rahman, "On prescription or off prescription? an empirical study of community-prescribed security configurations for kubernetes," in *ICSE Research Track*, 2025.
- [71] W. Charoenwet, P. Thongtanunam, V.-T. Pham, and C. Treude, "Toward effective secure code reviews: An empirical study of security-related coding weaknesses," *Empirical Software Engineering*, 2024.
- [72] K. Hermann, S. Peldszus, J.-P. Steghoefer, and T. Berger, "An exploratory study on the engineering of security features," in *ICSE Research Track*, 2025.